

Proposal: App Compatibility Matrix — Format and Maintenance

Proposed by: Atlas (operations-manager) **Date:** 2026-03-28 **Revised:** 2026-05-31 (CEO review comments applied) **Status:** Proposed **Parent:** [2026-03-28-app-dev-agent.md](#) (Kit agent proposal)

Problem

As Kit manages multiple App versions across multiple IDEA Engine versions, the question arises: which version of an App works with which version of the Engine? Without a machine-readable record of this, incompatible disks could be inserted into field Pis and fail silently (or loudly) — with no technician present to diagnose.

Two places need this information: 1. **Kit** — to know which Engine version to test against when building a new App release 2. **The Engine** — to warn when an incompatible disk is inserted

[app.yaml](#) — Single Source of Truth

Each app repo contains an [app.yaml](#) file that defines how the App is built, how each Service is monitored, and the App's compatibility with the IDEA Engine. This replaces the previously proposed [monitoring.yaml](#) — build configuration and monitoring strategy are inseparable and belong in the same file.

Service build approaches

Each Service within an App uses exactly one build approach (defined in [app.yaml](#)):

build value	Meaning
custom	Kit maintains a Dockerfile ; builds image on Pi from external resources
retag	Kit pulls a public DockerHub image and re-tags it under koenswings/
direct	compose.yaml references an upstream image directly (discouraged)

The **build** field drives both how Kit builds the image and what Kit monitors for new versions. Monitoring is a consequence of the build approach — not a separate concern.

Example: Kolibri (Approach A — custom Dockerfile)

```
# app-kolibri/app.yaml
app: kolibri
idea_app_version: "1.1"

compatibility:
  engine_min: "0.9.0"      # minimum Engine version this App Disk
                           # requires
  engine_tested: "1.0.0"  # Engine version Kit last ran tests
                           # against

services:
  - name: kolibri
    build: custom
    dockerfile: Dockerfile      # Kit-maintained; downloads
    Kolibri binary during build
    image: koenswings/kolibri
    monitors:
      - kind: http-scrape
        url: "https://learningequality.org/r/kolibri-latest"
        pattern: "kolibri-(?P<version>[\\d.]*)"
                                     # Kit monitors the external
resource (binary URL),
                                     # not a DockerHub tag – the
binary IS the build input
  - name: db
    build: retag
    upstream: mariadb
    upstream_tag: "10.6"
    image: koenswings/mariadb
    monitors:
      - kind: dockerhub
        repo: library/mariadb
        tag_pattern: "^10\\.\\.\\d+$"
```

Example: Nextcloud (Approach B — re-tag)

```
# app-nextcloud/app.yaml
app: nextcloud
idea_app_version: "1.0"

compatibility:
  engine_min: "0.8.0"
  engine_tested: "1.0.0"

services:
  - name: nextcloud
    build: retag
    upstream: nextcloud
    upstream_tag: "27-apache"
    image: koenswings/nextcloud
    monitors:
      - kind: dockerhub
        repo: library/nextcloud
        tag_pattern: "^\\d+-apache$"
  - name: db
    build: retag
    upstream: mariadb
    upstream_tag: "10.6"
    image: koenswings/mariadb
    monitors:
      - kind: dockerhub
        repo: library/mariadb
        tag_pattern: "^10\\.\\.\\d+$"
```

How Kit Uses `app.yaml`

On the daily monitoring cron run, Kit reads `app.yaml` for each App and applies the correct monitoring strategy per Service based on the `build` field:

- `custom` → Kit checks the `monitors` entries for new versions of external resources (e.g. scrapes a release page for a new binary). When a new version is found, Kit rebuilds the Docker image on the Pi from the updated `Dockerfile` and pushes it to `koenswings/<service>` on DockerHub.
- `retag` → Kit checks the DockerHub tags API for the `upstream` image. When a new matching tag is found, Kit pulls it, re-tags it as

koenswings/<service>:idea-<version>, and pushes it to DockerHub.

- **direct** → Kit checks the DockerHub tags API for the upstream image and updates the image tag in `compose.yaml`. No image build or push.

When Kit builds a new App release, it updates `engine_tested` to the current Engine version and runs tests. If the test harness finds a compatibility failure, Kit opens a `[From Kit] Feasibility` task on Axle's board before moving the task to review.

No Pex as a distinct build strategy. Pex files are simply an external resource that a custom `Dockerfile` downloads — they are covered by `build: custom` with an `http-scrape` monitor. The Dockerfile is what Kit maintains; the Pex URL is just the resource it monitors.

How the Engine Uses `app.yaml`

The Engine does not read `app.yaml` directly. Instead, `build-instance` (the Pi-side script that assembles an App Disk) reads `app.yaml` and writes a `META.yaml` onto the disk. `META.yaml` is what the Engine reads at disk-dock time.

```
# META.yaml (written by build-instance from app.yaml)
diskId: "abc123"
diskName: "Kolibri v1.1"
created: 17000000000000
app: kolibri
idea_app_version: "1.1"
requires_engine_min: "0.9.0"    # copied from app.yaml
compatibility.engine_min
```

When the Engine processes a newly inserted disk, it compares `requires_engine_min` against its own version. If the Engine is too old, it logs a warning and surfaces it in the Console UI rather than attempting to start incompatible containers.

This requires a one-time Engine change (Axle's task): read `requires_engine_min` from `META.yaml` and compare to the running Engine version. Kit creates a `[From Kit] Feasibility: requires_engine_min field` in `META.yaml` task on Axle's board to trigger this work.

build-instance — Where It Lives

The existing `build-instance` script currently lives in the engine repo. Since it is the script that packages App Disks — an app-level concern, not an engine concern — it should migrate to `app-harness` (Kit's shared build utility repo) as part of Kit's bootstrap work.

Atlas will raise this with Axle via a `[From Atlas] Review` task before Kit's onboarding begins, to confirm the migration path and ensure the engine repo does not break.

Maintenance

- Kit updates `app.yaml` in each app repo whenever a new Service version is released
 - `engine_tested` is updated on every Kit release run
 - `engine_min` is updated only when a new Engine feature is required by the App (rare; triggered by a `[From Axle] Review` task notifying Kit of a breaking engine change)
-

Impact

- **Kit:** owns `app.yaml` in each app repo; reads it for monitoring and builds; passes `compatibility` fields to `build-instance` to populate `META.yaml`
- **Axle:** adds `requires_engine_min` check to Engine disk-dock flow (one-time change; Kit opens a Feasibility task); confirms `build-instance` migration to `app-harness`
- **Each app repo:** gains an `app.yaml` (replaces any prior `monitoring.yaml`)
- **CEO:** compatibility warnings will appear in Console UI once Axle's change lands